Qwen 3.8 Vision OCR

Full-Stack AI Document Intelligence & Text Spotting System — Technical Overview

Author: Marco Yap (marcoyap41)
Project: Learning AI Engineering / OCR
Date: October 2026

Python 3.13 | FastAPI + Uvicorn | Groq API (qwen/qwen3.8-27b) | React 19 + Vite | Pillow (PIL) | uv Package Manager

1. Executive Summary & Motivation

This project builds a full-stack Optical Character Recognition (OCR) and spatial text-spotting web application powered by the **Qwen 3.8 27B Vision-Language Model** running on **Groq's LPU inference cloud**. Originally prototyped with local Hugging Face weights and a single-file Streamlit UI, the project was re-architected to overcome two major engineering bottlenecks:

- **Hardware & VRAM Bottlenecks:** Running multi-billion parameter vision models locally saturates consumer laptop VRAM (2 GB GPU) and system RAM. Offloading vision inference to Groq's API shifts heavy tensor computation to cloud LPUs while keeping local resource usage near zero.
- **UI/UX & Streaming Limitations:** Moving from Streamlit's full-script rerun model to a decoupled **FastAPI backend + React (Vite) frontend** unlocked real-time token streaming, interactive SVG bounding-box overlays, and live rolling rate-limit telemetry.

2. System Architecture

React + Vite Frontend
Drag/drop/paste image capture, zero-copy Blob preview, interactive SVG bounding-box layer, NDJSON stream reader, live rolling quota countdown & refill bar.

↔

FastAPI Backend
In-memory Pillow image pipeline, EXIF rotation fix, NDJSON `<think>` filter, coordinate normalization, 429 retry & JSON recovery.

↔

Groq Cloud API
`qwen/qwen3.8-27b` vision inference, SSE token streaming, and GCRA leaky-bucket rate-limit headers.

3. Backend Engineering & AI Reliability Pipeline (`backend/main.py`)

In-Memory Image Pre-Processing

- Reads uploaded bytes asynchronously into an in-memory `io.BytesIO` buffer without disk I/O overhead.
- Applies `ImageOps.exif_transpose` so rotated smartphone photos align with visual coordinates.
- Normalizes color channels (`RGBA` / `P` → `RGB`), clamps maximum dimensions to `2048px` , and encodes to Base64 JPEG.

NDJSON Streaming & `<think>` State Machine

- `POST /api/ocr/full` streams `application/x-ndjson` packets containing `rate_limit` , live `token` chunks, and final `usage` metrics.
- Implements a sliding-window state machine (`stream_events`) that detects and strips reasoning `<think>...</think>` blocks across chunk boundaries in real time.

AI Mitigation & Edge-Case Engineering

Challenge Encountered	Root Cause	Engineering Solution Implemented
HTTP 429 OTPM Rejection (`Limit 1000, Requested 2048`)	Groq's free tier enforces a 1,000 Output Tokens Per Minute (OTPM) limit and checks `max_completion_tokens` upfront.	Lowered `max_completion_tokens` to `900` and added automatic retry with regex limit extraction and dynamic token clamping on `RateLimitError` .
Bounding Box Misalignment on Portrait & Wide Images	`qwen/qwen3.8-27b` outputs coordinates scaled where the *longest image side = 1000* rather than per-axis 0–1000.	Calibrated `normalize_boxes()` using `scale_x = 1000 * W / max(W,H)` and `scale_y = 1000 * H / max(W,H)` to map boxes into `[0.0, 1.0]` ratios.
Truncated JSON & Duplicate Passes on Dense Images	When dense text hits the token limit mid-object or triggers a second pass, standard `json.loads()` fails completely.	Added regex object recovery (`re.finditer(r"\{[^{}]*\}", text)`) to salvage all completed boxes, plus out-of-bounds duplicate-pass filtering.
Spotting Accuracy Drop under Generic Prompts	Qwen-VL models are fine-tuned on specific grounding trigger phrases for line-level OCR detection.	Preserved Qwen's exact fine-tuned spotting prompt and `2048px` resolution while handling cleanup in post-processing.

4. Frontend Engineering & Interactive UX (`frontend/src/App.jsx`)

- **Multi-Modal Image Input:** Supports drag-and-drop, file selection, and global clipboard paste (`Ctrl+V`) with instant zero-copy preview via `URL.createObjectURL` and mid-flight cancellation via `AbortController`.
- **Resolution-Independent SVG Overlay:** Renders detected boxes inside an `<svg viewBox="0 0 1 1" preserveAspectRatio="none">` layer over the image. Hovering a bounding box highlights the corresponding text row (and vice versa), and clicking any box or row copies that line to the clipboard.
- **Live Rolling Quota Telemetry & Auto-Refill Timer:** Parses Groq's duration headers (`x-ratelimit-reset-*`) into milliseconds and runs a live 250ms ticker across four cards:
  - *Last Request Usage & Session Total:* Tracks per-request input/output token split, latency, and cumulative session tokens.
  - *Available Token Remaining:* Live remaining TPM quota (`8,000` limit) that smoothly refills and animates its progress bar back to `100%` as the token bucket replenishes.
  - *Available Request Remaining:* Live rolling 24h request counter (`1,000` limit) with a real-time countdown (`Refills to full in Xm Ys`) that automatically increments `+1` request every `86.4s` elapsed.
- **In-App Free-Tier Token Cap Notice:** Displays a clear notice banner informing users of the ~900 output token cap per request and recommending small-to-moderate text density to avoid mid-document truncation.

5. Key Achievements & System Limitations

Successful Measures Achieved

- **Zero Local GPU Load:** Reduced local memory footprint from ~15 GB (local 7B VLM) to <100 MB RAM while running a 27B parameter vision model.
- **High Spatial Accuracy (~1–2% Error):** Verified bounding-box alignment across portrait (`500x1200`), landscape (`800x300`), and wide (`1600x400`) aspect ratios.
- **Sub-Second Time-to-First-Token:** Real-time NDJSON streaming delivers extracted text immediately without blocking the UI thread.
- **Fault-Tolerant Parsing:** Regex object recovery prevents blank-screen crashes when outputs hit token caps mid-JSON.
- **Live Leaky-Bucket Observability:** Real-time countdown and automatic counter replenishment for both minute token pools and rolling 24h request pools.

Current Limitations (Free-Tier Constraints)

- **1,000 OTPM Output Ceiling:** Groq's `on_demand` free tier caps output for `qwen/qwen3.8-27b` at 1,000 tokens/minute (~900 max per request). Dense pages (40+ lines) can reach the token cap and truncate trailing lines.
- **8,000 TPM & 200k TPD Rolling Quotas:** High-resolution images consume ~700–1,400 prompt tokens each, limiting burst throughput to ~5–7 scans/minute. The 200k daily token pool refills continuously (~2.31 tokens/sec rather than a midnight reset).
- **1,000 RPD Rolling Request Pool:** Caps usage at 1,000 requests per rolling 24h window (refilling at 1 request per 86.4 seconds).
- **Single-Image Input:** Processes one raster image (JPG/PNG/WEBP) per request rather than multi-page PDFs.

6. Project Structure & Modern Tooling

File / Directory	Responsibility
`backend/main.py`	FastAPI server, Pillow image preparation, Groq async client, NDJSON streaming, coordinate normalization.
`frontend/src/App.jsx`	React UI state, stream reader, interactive SVG overlay, live rolling quota ticker, limit notice banner.
`frontend/vite.config.js`	Vite dev server configuration and `/api` reverse proxy to FastAPI on port 8000.
`app.py`	Legacy single-file Streamlit prototype preserved for reference.
`pyproject.toml` & `uv.lock`	Locked Python dependencies managed by `uv` (FastAPI, Uvicorn, Groq, Pillow, python-dotenv).